

Book Notes & To-Do

A running scratchpad for the eventual book adaptation of this course

VIDEO GAME CREATION

How To Use This Document

This is a working scratchpad, not a polished outline - it exists so that adding a thought while the course is being built doesn't require deciding, right then, where it "really" belongs. Two rules keep it useful instead of becoming a mess:

- Add freely. A new To-Do or Note can sit next to, or even contradict, something written earlier - resolve the contradiction later, not in the moment. Rethinking the whole document every time something gets added defeats the point of having it.
- Reorganize rarely, on purpose. Periodic dedicated consolidation passes (see the standing To-Do item below) are where duplicates get merged, stale items get cut, and contradictions actually get resolved - not as a side effect of adding one new line.
- To-Do items get updated or deleted outright once actually resolved - this section should always reflect current, real open work, not a historical log. Notes & Commentary is the opposite: it's allowed to accumulate, and old entries are kept (even superseded ones) until a consolidation pass deliberately prunes them.

To-Do

Open items, pulled from the project's task list and from things noticed while building other documents. Check one off or delete it outright once it's actually done - don't leave resolved items sitting here.

- ☐ Book version: move the Godot signals sidebar ("Godot Signals: Create It, Connect It, Write the Function, Emit It", with the four-piece diagram_godot_signal_four_pieces) from the Lesson 8 Sidebars to Lesson 6, where signals first appear (the destroyed signal for hits), and remove it from Lesson 8. Lesson 8 should then only point back to it, since escaped is the second use of the same four pieces. For the current course the sidebar stays in the Lesson 8 Sidebars document, since no Lesson 6 Sidebars document exists yet. Source: topic in build_sidebars.js (lesson08 array). *(decision by Robert; book retrofit only)*
- ☐ Rule 15 (explicit, beginner-friendly names) retrofit: Godot's Label-node names were renamed this session (WinLabel/LoseLabel/ScoreLabel/LivesLabel/WaveLabel -> WinText/LoseText/ScoreText/LivesText/WaveText, matching GDevelop's existing text

object names) after a student read WinLabel as Window Label. Audit every other engine and lesson for names that lean on engine jargon (Label, Node, Actor, Pawn, Controller, Template, Trigger, Behavior, Instance) or that a newcomer could misread, rename them to plain names, and keep the same plain name in every engine per Rule 12. Update the Data Dictionary and Cross-Engine Consistency Reference to match. Also check Robert's own test projects, which still use the old names. *(Standards Rule 15, added per Robert's direction; Godot text-node names done in Lessons 6-14, nothing else audited yet)*

- ☐ Script/Visual parity cleanup for the book: for engines that offer both a scripting and a visual path (GameMaker GML Code vs. GML Visual, Unity C# vs. Visual Scripting, Unreal C++ vs. Blueprint), make sure the two paths produce the SAME gameplay and appearance - same positions, sizes, text, colors, timing, sounds, and variable names - so a student can switch paths without the game changing. Robert's own GameMaker Code and Visual projects have drifted apart through experimentation, and the drift is easy to repeat whenever a function call and a block do not map one-to-one or the options are not obvious. Do a side-by-side pass lesson by lesson: list every function call next to its equivalent block with identical field values, flag where an exact equivalent does not exist and say so explicitly (Rule 4/7), and re-check after every content change. Consider a short parity checklist per lesson in the Prep Checklist, and verify against Robert's actual test projects, not just the documents. *(raised by Robert; GameMaker is the known problem case)*
- ☐ Book retrofit naming decision (Robert): all asset/object names in the book use PascalCase with NO common prefixes or suffixes (no obj_, spr_, snd_, sfx_ and the like), e.g. GameManager, Enemy, PlayerShip. Assets are kept organized with each engine's own mechanisms instead - folders in GameMaker's and Unity's asset browsers, groups, Scenes, Content Browser folders, and so on - so related assets stay together without name prefixes. This applies to the book retrofit only: for the current course pass, existing variable-naming conventions for each engine stay as already established, and the shipped GameMaker content (obj_gamemanager and friends) is left alone. When the book retrofit happens, apply it in one pass across all engines, update Rule 12 and the Data Dictionary, and decide whether Lesson 2 setup should teach creating the asset folders. *(decision by Robert; supersedes the earlier separate asset-prefix and capitalization To-Dos)*
- ☐ Final deliverable for BOTH the course and the book: a Complete Script Document per engine and coding style - the entire finished Semester 1 project in one place (every object, script, event, function, graph, or block sequence in full, organized by object in build order, with the lesson that introduced each piece noted). Generate it from the same lesson build scripts so it cannot drift. Needed because lessons now show only the methods they add or modify (Standards Rule 14). *(decision by Robert; Standards doc now lists it as a Final Deliverable)*
- ☐ Retrofit Standards Rule 14 (code completeness by level: Handout Level 1 complete code incl. initialization and display; Handout Levels 2-3 prose + diagrams; Teacher's Key complete code for all three levels) across Lessons 2-15 and all 8 engines. Lesson 8 GameMaker is the first case done (Handout Level 1 code box and Key Level 2/3 code). Known weak spot: most Keys describe Level 3 in talking-point prose only, and some Handouts show only the new lines. Audit lesson by lesson, starting with Lesson 8's other seven engines and Lesson 9. Scope note: Rule 14 requires COMPLETE code for every method a lesson adds or modifies (whole methods, not just the new lines), with the new lines highlighted; unchanged methods are not repeated, since the Complete Script Document (separate To-Do) covers the whole project, because showing only new code has already confused students. Lessons 8-15 are brought up to this standard as each is reviewed; Lessons 2-7 are deliberately deferred to the book retrofit (they currently show

new code only), and Lessons 16-30 must follow it as authored. *(Standards Rule 14, added per Robert's direction; only Lesson 8 GameMaker complete; Lessons 2-7 deferred to the book retrofit)*

- ☐ Finish the GameMaker __dnd_lives / __dnd_score review for Lessons 10-15 (restart/lives reset and any other lives/score use): replace deprecated built-in lives/score and "Set Instance Variable ... Lives/Score" with the purpose-built Lives/Score blocks (Target = obj_gamemanager) in Visual paths and __dnd_lives/__dnd_score in code paths. Deliberately held until Lessons 8 and 9 are fully reviewed. *(added after Lessons 8-9 GameMaker corrections; Lessons 10-15 not yet reviewed)*
- ☐ GameMaker: review every use of obj_name.variable notation in code paths and Visual paths. Where the script runs on the target object itself (or a block has a Target drop-down), prefer the plain variable name plus the Target drop-down over the dotted form, and make the code path and Visual path teach the same idea. Needs a deliberate pass across all lessons, not piecemeal edits. *(raised by Robert while reviewing Lessons 8-9)*
- ☐ GameMaker sidebar and comparison test: confirm that (a) the If Lives block with Target = GameManager, (b) an If Variable/Expression block testing __dnd_lives with Target = GameManager, and (c) a code if-statement using obj_gamemanager.__dnd_lives all behave identically, and write it up as a Sidebar (three ways to ask the same question about the same instance variable). Run the test in GameMaker first and record actual results before writing. *(raised by Robert; needs hands-on GameMaker testing)*
- ☐ Cross-engine design drift (HUD object, prefabs, helper objects): during the Lesson 10 review Robert noticed that structural pieces introduced in one engine are missing or different in others. Examples: Unity and Unreal got a dedicated HUD (a HUD GameObject / WBP_HUD widget) while Scratch, Godot, GameMaker, GDevelop, TIC-80 and 001 Game Creator draw score and lives directly from the GameManager or from labels; Unity uses Prefabs for enemies while GameMaker's equivalent was never used even though one exists. Before the book, audit every engine for pieces present in some but not all (HUD, prefab/template for enemies/bullets, spawner, restart routine, start_game function), decide per piece whether it is a course-wide concept (then add it to every engine, using each engine's native equivalent) or an engine-specific need (then document the driving reason per Rule 5), and record the result in the Cross-Engine Consistency Reference and Data Dictionary. Add a Standards check: any new structural object introduced for one engine must be introduced for all, or the reason documented. *(raised by Robert at Lesson 10 review; audit needed)*
- ☐ Cross-engine name normalization (generalized): the same object, variable and function must be spelled identically in every engine and every document wherever the engine allows, and the casing must follow the single book convention (PascalCase for assets and objects, per the naming decision above). Known example found at the Lesson 10 review: GameMaker's GameManager object is spelled obj_gamemanager in the shipped handouts but obj_GameManager in Robert's own test project (drift from the late switch to PascalCase). In the book pass, audit and normalize every name (GameManager, game_active, enemies_remaining, lives, score, start_game and the rest) against the Data Dictionary, per engine, then regenerate all Handouts, Keys, Slides, diagrams and the Complete Script Documents from the normalized names. *(raised by Robert at Lesson 10 review; book pass)*
- ☐ Canonical bug-hunt exercise (decision): the win/lose gap (kills reduce enemies_remaining but escapes only reduce lives, so some rounds end with no win and no loss, and R does nothing) is INTENTIONALLY left in Lessons 6 and 8. It is the Lesson 10 Level 3 'Fix a real bug!' task and part of the canonical lesson plan, also for the book. Do not fix it earlier. Plan at least one similar find-and-fix-your-own-design-bug

exercise in Semester 2 (Lessons 16-30), where the groundwork is laid in earlier lessons by thinking about two cases separately instead of as one state. *(decision by Robert at Lesson 10 review)*

- ☐ Reconsider static members in Unity's GameManager (Robert): from Lesson 6 on, the Unity GameManager keeps shared state in static fields and static methods (RegisterKill, RegisterEscape, score, lives, gameActive). That is more complicated than the other engines need and forced a Singleton in the first Lesson 10 draft, since a static method cannot use an Inspector reference. Lesson 10 now avoids it with an ordinary instance method ResetGame() plus an Inspector slot for the spawner. For the book, decide whether GameManager should be a normal component with instance fields (other scripts reach it through an Inspector reference, as with the enemy prefab), and retrofit Lessons 6-10. Static members and singletons may then be a good Semester 2 refactoring topic, introduced on purpose. *(raised by Robert at Lesson 10 review; book pass)*
- ☐ Unity GameManager without statics AND without Find...() (book design change, related to the static-members item above): in the book version, avoid FindFirstObjectByType and similar lookups as well as statics. The obstacle is that a runtime-spawned prefab (Enemy, Bullet) cannot hold an Inspector reference to a scene object. Robert believes he solved this in an earlier project: REVIEW HIS OLD PROJECTS for a beginner-level approach before choosing one. Candidates to compare against it: (1) the spawner holds the GameManager reference in its own Inspector slot and hands it to each new enemy right after Instantiate; (2) the GameManager does the spawning itself, so it already holds every reference; (3) events/UnityEvents, the Unity cousin of the Godot signals sidebar (create it, connect it at spawn, handle it on GameManager, invoke it); (4) a shared ScriptableObject holding the game state. Pick the one with the least new vocabulary that fits Lessons 6-10, record the decision, and update Rule 12 and the Data Dictionary so the Unity design matches the other engines. *(raised by Robert at Lesson 10 review; review old projects first)*
- ☐ Design principle, reiterated (cross-engine drift): the game design should mirror itself from engine to engine as much as possible: same objects, same names, same state on the same manager, same flow, same behavior on screen. The one exception is where an engine fundamentally refuses to do it, or where mirroring would need an excessive, unnecessary workaround. In that case the engine's own idiom overrules consistency, and the deviation is recorded with its driving reason (Standards Rules 5 and 12). Apply this test to every item in the drift family: the HUD object, prefabs, the Unity statics and Find...() question, Godot signals versus other patterns, and the Scratch clone and broadcast handling. When one engine needs a heavier pattern than the others (the Lesson 10 Unity Singleton was the example), look first for a simpler design that all engines can share before accepting the deviation. During the book pass, fold this wording into Rule 12 and keep a per-engine list of accepted idiom exceptions in the Cross-Engine Consistency Reference and Data Dictionary. *(raised by Robert at Lesson 10 review; principle for the book pass)*
- ☐ Objects talking to each other: signals, broadcasts and events (add to the Unity design reconsideration above): Godot signals have caused difficulty and blank stares, and the course never gave a proper discussion of what a signal IS before using one. Scratch's Broadcast was glossed over the same way, and Unity events (candidate 3 above) would raise the same problem. For the book, treat 'one object announces something, another object reacts' as a named concept introduced on purpose, before it is first used (Lesson 6 or earlier, matching the signals-sidebar move already logged), with the same vocabulary and the same picture across engines: Scratch broadcast and receive, Godot signal and connect, Unity event, GameMaker direct calls. Also compare simpler Godot

alternatives before settling on signals: the Enemy calls a GameManager function directly through a node path, a Group call (get_tree().call_group), or an Autoload GameManager (Godot's counterpart of the static idea above, so decide it together with the Unity static question). Robert is not sure Godot has a simpler alternative, so verify these against a real project. Pick one pattern per engine, record the reasons, and add the picture to the Rule 9 diagram set. *(raised by Robert at Lesson 10 review; design decision plus a new concept lesson)*

- ☐ Unity static GameManager - revisit in THIS course if needed: the Lesson 6-12 Unity code keeps its static GameManager for now. If a student struggles with it, add a 'Non-static variant' box to the Unity Keys: remove static from the fields and from RegisterKill/RegisterEscape, and in every script that uses GameManager (Enemy.cs, Bullet.cs, the Lesson 12 sound code and so on) add a GameManager field filled in Start() with FindFirstObjectByType<GameManager>(), then change each GameManager.something call to use that field. Watch for the errors 'An object reference is required for a non-static member' and NullReferenceException. *(decision by Robert at Lesson 10 review; do only if needed)*
- ☐ Active-engine scope (set at the Lesson 10 review): the only engines students are currently using are Scratch, Godot, GameMaker (script and Visual) and Unity (script). Until the book pass, new and revised lesson work covers only those. Deferred, not dropped: Unreal, 001 Game Creator, Unity Visual Scripting, and GDevelop and TIC-80 (confirmed paused by Robert). For the book pass, catch each deferred engine up on every Rule 11-15 change, the Lesson 10 restart design, and the Rule 9 diagrams, using the shipped Lesson 10 content (Unreal and 001 Game Creator were already updated in Lesson 10 before this decision) as the starting point. Rendering and re-shipping the deferred engines' Handouts/Keys is paused until then. *(raised by Robert at Lesson 10 review; book pass)*
- ☐ Unity Visual Scripting pass (deferred): no students are currently using the Unity Visual Scripting path, so it was left out of the Lesson 10 pass. For the book, review every Unity Visual section from Lesson 8 on: verify node names against a real Unity install (e.g. the Lesson 10 restart graph and its On Input System Event Button node), apply the Rule 13/14 changes (state on the manager, complete code or graph per level), and build the missing Rule 9 Unity Visual node diagram for Lesson 10 (Lessons 7-9 already have one). *(raised by Robert at Lesson 10 review; book pass)*
- ☐ Lesson 10 compliance pass (done): Handouts and Keys for all eight engines now follow Rules 11, 13, 14 and 15 (shared state on the manager object, explicit loop/counter names, complete highlighted Level 1 code, Key code boxes for Levels 1-3). Rule 8 (slide speaker notes copied from the Teacher's Script, plus a new Activity Diagram slide) and Rule 9 (restart-flow Activity Diagram in all Handouts/Keys/Slides; GameMaker Visual and Unreal Blueprint node diagrams) are also done. Remaining: the GameMaker Visual R-key block is confirmed as 'If Key Pressed' (Letters > R), and the Unity Visual path is deferred (see above); Lessons 11-15 still need the same Rule 13/14 pass. *(Lesson 10 review)*
- ☐ Reflex habits: testing, documentation and refactoring as things students do without thinking of them as separate tasks (Robert's goal, raised at the Lesson 11 review after his own GameMaker project drifted from the Keys): see the separate document Reflex_Habits_Design_Notes.docx (this folder). It holds Robert's problem statement, Claude's first-pass commentary (yes at beginner level if taught as three plain habits: try it, say it, tidy it, in the last five minutes of lab), a draft candidate Rule 16 (Done Means Tried, Named and Tidy), concrete mechanisms, options for fixing the instructor's own drift, a long spitballing list, risks, and the decisions needed. Nothing in it is adopted yet. *(raised by Robert at the Lesson 11 review; decision pending on candidate Rule 16)*

- ☐ Name the bug: "the Stalemate" (book edition, Robert's decision at the Lesson 11 review): refer to the win/lose gap bug by this name everywhere in the book and course materials from now on. Definition for the manuscript: a round that ends with no win, no loss and no restart, because escapes lower lives but never lower enemies_remaining. The name should appear the first time the bug is reported (Lesson 10's Level 3), be used in Lesson 11's wave discussion, and in any Semester 2 repeat of the bug-hunt exercise. The name also appears in Lesson 10 and Lesson 11 handouts and Teacher's Keys as of Lesson 11 (Lesson 10's own Level 3 text does not name it yet, only the Lesson 11 materials do; decide whether to add the name to Lesson 10's Level 3 in the book). *(decision at the Lesson 11 review; name adopted from Robert)*
- ☐ Lesson 11 compliance pass (done for active engines): Handouts and Keys for Scratch, Godot, GameMaker (script and Visual) and Unity were rewritten to match the Lesson 10 design: wave and total_waves live on the GameManager (Rule 13), the Lesson 10 spawn routine is reused as the 'spawn a wave' routine (spawn_enemies / SpawnEnemies, broadcast [spawn enemies]), complete highlighted Level 1 code (Rule 14), Key code for Levels 1 to 3 (Rule 14), Activity Diagram (new, diagram_activity_wave_flow) and a GameMaker Visual node diagram (Rule 9), game-state check for the new state, speaker notes copied verbatim from the Teacher's Script (Rule 8), Unity demo rewritten without the singleton (Rule 10). The Stalemate is deliberately left in for students to find and fix (Level 3), with a worked wave-aware fix in each Key. Paused engines (Unreal, 001 Game Creator, GDevelop, TIC-80, Unity Visual) were NOT rebuilt: their Lesson 11 files in the lesson folder are the old, stale versions (singleton, spawn_wave naming, no Rule 14 code) and must be rebuilt from the active-engine design during the book pass. *(Lesson 11 review, Rules 8, 9, 10, 11, 12, 13, 14, 15)*
- ☐ Unity static GameManager - concrete cost found in Lesson 11: because RegisterKill() is static, it cannot use the instance Spawner slot, so the wave request is passed through a static flag (nextWaveRequested) that GameManager's Update() acts on. It works and teaches 'static methods cannot reach instance members', but it is exactly the workaround the non-static redesign (see the static-members items above) would remove: with an instance GameManager, RegisterKill() would simply call spawner.SpawnEnemies(). Use this as the motivating example in the book's static-versus-instance discussion. *(found while rebuilding Lesson 11 Unity, Lesson 11 review)*
- ☐ Verify at the keyboard (Lesson 11, not tested by the author): (1) Godot: the next wave is requested with get_parent().call_deferred("spawn_enemies") because a wave clears inside a collision callback (suspected 'flushing queries' restriction on adding physics objects); confirm whether the deferral is actually required or merely harmless, and update the explanation accordingly. (2) GameMaker Visual: the Else block placement after a nested If Variable, and the Draw Value block's field labels, were written from the earlier Lesson 8 to 10 block names and have not been clicked through. (3) Scratch: confirm the last-hit clone never double-spawns a wave (it deletes itself before GameManager's broadcast arrives). *(Lesson 11 review, awaiting Robert's testing)*
- ☐ GameMaker Prefabs: GameMaker has a Prefab concept. Unity introduces Prefabs in the course, so decide where and how GameMaker's equivalent is introduced (likely alongside the lesson where Unity first uses Prefabs), verify what GameMaker Prefabs actually are and how they differ, and update the cross-engine consistency reference and Data Dictionary accordingly. *(raised by Robert; research needed)*
- ☐ Resolve or explicitly caveat GameMaker's Lesson 13 explosion scale-origin risk before that lesson is written up. *(Task #243, still open)*

- ☐ Finish auditing Unreal Lessons 5, 6, 8, 9, 11, and 14 for the same missing "Split Struct Pin" step found (and fixed) in Lessons 2, 3, and 7. *(Task #254, partially resolved)*
- ☐ When describing Lesson 14 in the manuscript, explicitly call out that its single free-choice Level 3 (breaking the Sound/Visual/Structure template used every lesson since Lesson 2) is intentional scaffolding removal, not an inconsistency. *(found while building Cross_Engine_Consistency_Reference.docx)*
- ☐ Get Robert's decision on 001 Game Creator sprite import steps before that engine's chapter/section can be considered complete. *(Task #209, waiting on Robert)*
- ☐ Before claiming full 8-engine coverage anywhere in the book, finish QA-validating every engine in every mode (visual/script/block), not just the current focus subset (Scratch, Godot, Unity Scripting, GameMaker Scripting, GameMaker Visual, Unreal). *(Task #249)*
- ☐ Revisit lesson pacing across Lessons 3-15 once the full semester has actually run with students - note anywhere the real classroom pace diverged from the written plan before treating the lesson plans as the book's final word. *(Task #119)*
- ☐ Decide whether/how the course's companion digital materials (Handouts, Keys, GDDs) get referenced or reproduced alongside the book - affects how self-contained the book needs to be. *(Task #128, still open)*
- ☐ Check whether IGDA's Game Education SIG has released their updated Curriculum Framework yet, and reconcile the book's framework-alignment claims against it if so - the version available at project start (v3.2 beta, 2008) was already old, and an update was rumored to be in progress. *(see custom project instructions)*
- ☐ Periodically review and consolidate this document's Notes & Commentary section - merge duplicates, resolve contradictions between earlier and later notes, cut anything that turned out not to matter. Do NOT do this reflexively every time a note is added; batch it into occasional dedicated passes. *(standing item - see How To Use This Document)*
- ☐ Write a Lesson 4 Sidebar (or a Rule 6 Refinement and Style section, once that's a course fixture) on "who's in charge" - Bullet and Enemy each detecting their own hit and destroying only themselves, instead of one side reaching into the other. Ties to a live classroom discussion already introduced verbally but not yet formalized in writing. *(raised while checking Unity Lesson 4 sound handling, deliberately deferred rather than retrofitted)*
- ☐ Decide whether the 15 already-completed lessons get a retrospective retrofit pass for Standards Rule 6 (SuperStretch tie-in, Refinement and Style, visual-scripting node-reuse), or whether that content is written fresh for those lessons only when the book manuscript reaches them. See Notes & Commentary for the tradeoffs and a leaning recommendation - not yet decided. *(Standards Rule 6, added this session; scoping only, no retrofit started)*
- ☐ When rebuilding/auditing the Unreal Lessons, verify the Setup instructions actually include creating a no-Pawn Game Mode Blueprint (parent Game Mode Base, Default Pawn Class set to None in its Class Defaults) and setting it as the project's Default Game Mode - and check World Settings > Game Mode Override on each lesson's level too, since an override there takes priority over the project default. Confirmed missing/lost from Robert's own reference project this session: without it, the engine auto-spawns and possesses a stray built-in DefaultPawn at BeginPlay, which caused a hard-to-diagnose symptom (a black sphere rising out of view, tied to Space being a legacy default "move up" binding on DefaultPawn colliding with Fire also being bound to Space) plus spurious collision-notice firings. *(confirmed via Robert's hands-on Unreal rebuild this session; see also the DefaultPawn note already under Notes & Commentary)*

- ☐ Reminder for Robert, not a course-content item: once Lesson 05 testing starts on his own project, go back and add Lesson 4's Color Vector Parameter tint / Lesson 5's striped-row idea into his own per-engine test projects - he skipped it during Lesson 4 testing as optional/untested, but confirmed (by reading the actual Lesson 4/5 Unreal content) that it's genuine forward-designed setup, not filler. *(personal test-project follow-up, not a document edit)*
- ☐ Retrofit past lessons' visual-scripting alternate-path Handouts and Teacher's Keys against Standards Rule 7 (name every block, category, and field - not just the shape of the logic). GameMaker's Lesson 5 GML Visual altPath was the first case caught and fixed, while testing his own reference project surfaced two real gaps: the setup step didn't distinguish deleting a room instance from deleting the object asset, and the altPath's note wrongly implied the Level 3 color-switch idea needed no changes from the code path when it actually needed its own block sequence. Audit every other lesson's GML Visual, Unity Visual Scripting, GDevelop, 001 Game Creator, and Unreal Blueprint alt-path content the same way - unlike Rule 6, Robert has directed this one to be retrofitted, not deferred to book-writing time. *(Standards Rule 7, added this session per Robert's explicit direction; Lesson 5 GameMaker, Lesson 8 (GameMaker/Unity altPath field/node explicitness), and Lesson 9 (Unity Visual Scripting altPath node-name explicitness) done, Lessons 2-4, 6-7, and 10-15 not yet audited)*
- ☐ Retrofit Lessons 1-4, 7, and 10-15's existing Slide Decks (.pptx) with Teacher's Script content in PowerPoint speaker notes, per Standards Rule 8. Lesson 5's deck already has this (each content slide's addNotes() carries that slide's matching Teacher's Script section verbatim, including bracketed stage directions) and is the reference example to follow; Lessons 6, 8, and 9 now also have full coverage (each lesson's six new notes calls follow the same slides-5/7/9/11/13/14 pattern Lesson 6 set, skipping title/overview/divider slides). Robert's classroom now runs Presenter View across two displays, so the notes pane doubles as his live script during lecture - this is a genuine workflow need, not just a nice-to-have, but Robert explicitly deferred the retrofit rather than asking for it done immediately. *(Standards Rule 8, added this session per Robert's explicit direction; Lessons 5, 6, 8, and 9 done, Lessons 1-4, 7, and 10-15 not yet started)*
- ☐ Retrofit Lessons 1-4, 6, and 10-15 against Standards Rule 9 (diagrams): add the generic Activity Diagram to every already-shipped Handout, Key, and Slide Deck; add engine-specific node diagrams to GameMaker/Unity/Unreal's Handouts (Level 1, unconditional) and Keys (Level 1 always, plus a combined Level 3 sequence diagram where the visual path is complex enough to warrant one); and add diagrams to existing Sidebars where they'd genuinely clarify the text, per the same rule's Sidebars/Slides guidance. *(Standards Rule 9, added this session per Robert's explicit direction; Lessons 5, 7, 8, and 9 now have this treatment, Lessons 1-4, 6, and 10-15 not yet retrofitted)*
- ☐ Apply Standards Rule 9's diagram standard to Lessons 16-30 as each is authored, the same way it's now required for Lesson 5 onward - don't let the not-yet-built future lessons slip back to the pre-Rule-9 no-diagram default. *(Standards Rule 9, forward-application reminder)*
- ☐ Expand the Lessons 5-14 lecture-demo rotation (Standards Rule 10) from its current 5-choice pool (Scratch, GameMaker script, GameMaker visual, Unity C#, Godot) to evenly cover all 8 engines and all 3 coding styles, once students are actually using the currently-unused engines (GDevelop, TIC-80, 001 Game Creator, Unity Visual Scripting, Unreal Blueprints - none have active students right now). The original 30-Lesson Master Schedule already models what full-diversity balance looks like and can be adapted rather than redesigned from scratch. Until then, the restricted 5-choice rotation is intentional, not an oversight. *(Standards Rule 10, added this session per Robert's explicit direction to narrow the pool for now)*

- ☐ Retrofit past lessons' code and visual-scripting content against Standards Rule 11 (meaningful variable names over generic i/j/k counters), across all engines and coding styles. Lesson 5's audit is now complete across all 8 engines: GameMaker (Code and Visual paths, i -> enemy_count, inst -> new_enemy), Godot (Code path, i -> enemy_count, e -> new_enemy, plus a ternary -> if/else cleanup), Unity (Code path, i -> enemyIndex, e -> newEnemy, ternary -> if/else), and TIC-80 (Lua, i -> enemy_count in the spawn loop only - the separate bullets-iteration and enemies-array-iteration loops use i/j for a different concept and were deliberately left untouched) all needed and got a rename. Unreal's C++ alt-path (i -> EnemyCount) also needed one; its Blueprint path needs none (the bare "Index" it uses is the ForLoop node's own fixed pin name, not a variable this course chose - same as Unity Visual Scripting's Index pin). GDevelop and 001 Game Creator were both audited and found to need no change, for two different reasons: GDevelop's self-declared EnemyIndex scene variable was never generic to begin with; 001 Game Creator has no loop or variable at all for this concept - Level 1 is pure manual copy/paste placement in the Scene editor, and Level 3's row variety is manual per-instance Tint editing, so there's nothing to rename. Lesson 5 is done; every other lesson (1-4, 6-15) has not yet been audited for the same generic-counter pattern. Robert has adopted a standing workflow for this going forward: rather than a dedicated retrofit sweep, run a quick Rule 11 naming pass on an engine's Lesson 5+ content the next time he personally starts testing that lesson/engine combination, opportunistically, as he works through the course. *(Standards Rule 11, added this session per Robert's explicit direction; Lesson 5 fully audited across all 8 engines (GameMaker/Godot/Unity/Unreal C++ renamed, Unreal Blueprint/GDevelop/001GC confirmed no change needed, TIC-80 renamed), opportunistic per-lesson-as-tested workflow adopted for Lessons 1-4/6-15 in place of a dedicated retrofit sweep)*
- ☐ Pick a specific Semester 2 lesson to formally introduce the ternary operator, once Semester 2's actual lesson content is being built - Robert's own suggestion, prompted by pulling the ternary out of Lesson 5's GameMaker code for being too advanced this early. Not yet scheduled to a lesson number; needs to land somewhere a student is already comfortable with plain if/else and is extending an existing codebase, not meeting conditionals for the first time. *(raised alongside the Lesson 5 ternary removal; see the matching Notes & Commentary entry)*
- ☐ Rename Scratch's Lesson 5 loop variable from enemy_number to enemy_count, to match GameMaker and Godot. Robert's decision: enemy_count wins the naming question outright (it already had two engines using it before this was even raised as a question), so Scratch - the outlier, and the only engine using enemy_number - gets reconciled to match rather than the other two being changed. Not yet done as of this entry. *(Robert's explicit decision, given while Unity's Rule 11 retrofit was underway; supersedes the earlier not-yet-decided framing of this same question)*
- ☐ Run a dedicated, coordinated pass reconciling BOTH variable/function naming AND code/graph structure across all 8 engines and all 3 coding styles, per new Standards Rule 12 - broader in scope than the existing opportunistic, per-lesson-as-tested Rule 11 naming workflow (above), which only catches naming, one engine at a time, as Robert happens to test it. This pass should also resolve every deliberately-deferred word-choice and structural conflict surfaced along the way rather than fixing any one of them in isolation. Known cases so far, all in Lesson 5's spawn-loop concept alone: Unity's enemyCount (existing total-to-spawn field) vs. enemyIndex (the Rule-11-compliant but non-matching loop counter name used to avoid shadowing it); Scratch's enemy_number vs. the enemy_count used everywhere else (Robert's decision: enemy_count wins, tracked as its own ToDo above); GDevelop's EnemyIndex, meaningful but a different word choice ("index") than the "count" used by GameMaker/Godot/Scratch-once-renamed; and Unreal Blueprint's spawned-actor reference, which is never promoted to a

named variable at all (wired directly from Spawn Actor from Class's Return Value), unlike every other engine's named new_enemy/newEnemy - a structural gap, not just a naming one. This pass should also settle the open PascalCase-vs-camelCase-vs-snake_case question once and for all (see the separate ToDo item on that question) so the coordinated naming pass has one settled convention to apply rather than re-litigating case style engine by engine. *(Standards Rule 12, added this session per Robert's explicit direction; a fundamental precept (common ground across engines so students can help each other regardless of engine/style), not yet started; growing list of known reconciliation cases tracked here as they're found)*

- ☐ Decide the course's cross-engine variable/function naming CASE convention (PascalCase vs. camelCase vs. snake_case vs. some other scheme) as part of the Standards Rule 12 reconciliation pass (above) - a distinct question from which underlying WORD an engine uses (e.g., enemy_count vs. enemy_number), which is about the letter-casing/formatting style layered on top of whichever word is chosen. Robert's stated inclination: favor cross-engine consistency in the actual case/format used, even in engines where a different case convention is the idiomatic local norm (e.g., Unreal/C++ typically PascalCase, C-family languages typically camelCase, GML/GDScript typically snake_case) - and where the chosen course-wide convention deviates from an engine's own idiom, explain that deviation explicitly in a Sidebar rather than silently picking the engine-idiomatic case and letting the inconsistency stand unexplained. Not yet decided which specific convention wins; this entry only records the framing and Robert's leaning, not a final choice. *(raised alongside the Data Dictionary discussion; Robert's explicit inclination given, final convention choice deferred to the Rule 12 pass)*
- ☐ TIC-80's bare loop variables j and e (the enemy-array iteration pattern, e.g. "for j=1,#enemies do" / "local e=enemies[j]") were renamed to enemy_index/enemy for Lessons 7 and 8 as part of each lesson's own compliance pass. The identical unrenamed pattern was confirmed, by direct grep, still present in Lessons 5 and 6's build scripts. Robert's lesson-scoped authorizations for those passes didn't extend to Lessons 5-6, so this was deliberately left as a follow-up rather than done opportunistically alongside them. Fold into either a dedicated Rule 11 TIC-80 sweep or the broader Rule 12 reconciliation pass (above), whichever comes first. *(found during Lesson 7's Rule 13/diagram compliance pass, then confirmed already fixed for Lesson 8 during that lesson's own pass; Lessons 5-6 still open)*
- ☐ Standards Rule 13 (dedicated manager object for shared/global state) has only been retrofitted into Lessons 6, 7, 8, and 9 so far (Scratch, Godot, GameMaker, and Unreal gained a GameManager-equivalent object; Unity already had one, which is what prompted the rule; Lesson 8 additionally retrofitted the lives/has_lost lose-condition state onto the same object, and Lesson 9 retrofitted score the same way across GameMaker, Scratch, Godot, and Unreal). Lessons 10-15 haven't been audited yet for the same pattern - any lesson whose Handouts/Keys have a spawner, player, or other object directly owning a shared counter (a win/lose flag, a score, a lives count) rather than routing it through a dedicated manager object is a Rule 13 candidate. Audit opportunistically, the same adopted workflow as Rule 11's naming retrofit, rather than as a dedicated sweep, unless Robert says otherwise. *(Standards Rule 13, added this session; Lessons 6-9 done, Lessons 10-15 not yet audited)*
- ☐ For the book manuscript, move the GameManager mini-lecture ("Before We Start: Meet the GameManager" / "Why a Dedicated Manager Object?") from where it's live-taught to where it actually belongs conceptually: Lecture 06, not Lecture 07. The two slides live in Lesson 7's deck right now purely for production-timeline reasons - the retrofit was designed and caught after Lesson 6 already shipped, so it had to land as a lab-time catch-up notice in the next lesson taught, not the lesson the object was actually introduced in. The book isn't bound by that same live-classroom constraint, so this

content should be written into Lecture 06's chapter instead, presented as part of introducing the win condition and shared state in the first place, with Lesson 7's chapter only needing a brief backward reference rather than the full explanation. Keep the two slides in Lesson 7's live deck as-is; this is a book-only reorganization, not a live-course change. *(raised alongside adding the GameManager mini-lecture slides to Lesson 7's deck; book-only reorganization, live deck unaffected)*

- ☐ New companion book/class concept, spitballed, not scoped or scheduled: "Debugging 101" - a course/book built around a pre-loaded code repository seeded with a variety of deliberately planted bugs, modeled directly on real categories this very project has hit: flat-out broken code that doesn't compile/run, failed edge cases, a new feature that breaks an existing one, a mismatch between the design doc and the actual code, a broken refactoring, and a misinterpreted feature (built to spec as written, but not to what was actually meant). Natural companion idea: "Testing 101" - the inverse situation, an existing codebase plus a design doc but NO test cases, where the student's job is to build the test suite from scratch. A possible third, combined course: "Testing/Debugging 201", merging both - use the act of writing test cases as the mechanism that both finds the bugs AND proves each fix actually worked (red test before the fix, green test after), rather than teaching testing and debugging as separate skills. All three are genuinely downstream of THIS project specifically: the bug categories map directly onto real bugs found while building this course (see the Notes & Commentary entries on the spawn-loop self-collision bug, the GameMaker Visual targeting saga, the dropped Level 3 sounds, the WaveAnnouncementText/WaveText naming mismatch, and the Stalemate itself), which is exactly the kind of real-world-grounded material this idea would need. Not scoped, not scheduled, no engine/language/audience decided yet - purely a note-it-before-it's-lost entry; see the matching Notes & Commentary entry for the fuller framing. *(Robert's idea, spitballed; companion concept to the main Video Game Creation course, not yet scoped)*
- ☐ GameMaker GML Visual audit: confirm whether any existing Visual-path content assumes an "Else If" node/block exists. It appears GML Visual has no such block - a three-or-more-way branch (if / else-if / else-if / else) has to be built by nesting a second If Variable (or equivalent) block inside the first one's Else slot, one level deeper each time, rather than chaining same-level blocks the way code's else if does. Any Handout, Key, or diagram instructing a student to "add an Else If block" in GML Visual is describing a block that may not exist and needs rewriting as nested If/Else instead; any node diagram showing a flat else-if chain for GameMaker Visual needs the same correction, drawn as actual nesting. Not yet audited against the shipped lessons - verify at the keyboard which lessons' GML Visual content has a 3-plus-way branch (Lessons 8-9's lives-vs-enemies-remaining check and Lesson 11's wave-cleared check are the most likely candidates) and confirm whether they already nest correctly or need fixing. *(raised by Robert; verify at the keyboard before concluding which lessons are affected)*
- ☐ Extend Standards Rule 12 (cross-engine consistency) to cover pop-up/overlay text presentation, not just variable and function naming: wave announcements, win/lose banners, and other HUD or momentary-message text should look as similar as reasonably possible from engine to engine - specifically including things like centering on screen, vertical placement, and text size, which the course has so far let drift silently (Robert's own words: "I've silently deviated on a few things like text placement, size, and such"). Proposed shape of the fix: define one simple, explicit set of reasonable defaults (e.g., centered horizontally, a consistent vertical band for a given message role, a consistent relative size for "HUD counter" text vs. "big announcement" text) and apply it as the baseline in every engine's Handout/Key, while still leaving room for a student to deviate on these small things if they want to. Robert's explicit design goal for how this

should read in the lesson materials: it should take very little callout in the lesson plans themselves - the consistency should mostly just BE there, quietly, rather than being taught as its own topic - while still being fairly obvious to a student who looks at the actual steps what the convention is and why those particular values were chosen. A secondary benefit Robert flagged on purpose: nailing down one specific way to do a small, otherwise-you'd-have-to-look-it-up task (centering text on screen in each engine, for instance) functions as a tiny, free lesson in that engine's own text/UI API that the course doesn't have to spend any lecture time on, but that saves a student from going and searching for it themselves. Needs: an actual audit of every existing pop-up/HUD text element across the shipped lessons to find where defaults have already silently diverged, a decided set of default values per message role, and a lightweight way to state the convention in each engine's Setup/instructions without it reading as a big teaching moment. *(raised by Robert; extends Rule 12's existing scope from naming/structure to visual presentation of HUD/overlay text; not yet audited or decided)*

Notes & Commentary

Freeform observations worth remembering for when the book itself gets written - narrative angles, good anecdotes, structural ideas, and things that only became obvious by actually building the course. Seeded initially by pulling from the project's history up through the Unreal Lesson 2/3/4/13 rebuild and the Cross-Engine Consistency Reference document.

GML Visual: Set Instance Variable can't target a dynamically-created instance: Robert tested Lesson 5's GameMaker Visual color-switch instructions hands-on and found the originally-written steps didn't work: they called for dragging Create Instance's output into a "Set Instance Variable" block's Target field, but that block's targeting dropdown only actually offers Self, Other, or a specific object asset (which recolors every existing instance of that object, not just the newly spawned one) - there's no option to target an instance held in a variable at all. The fix he found and confirmed working: give Create Instance its own Target field (a new local variable, e.g. new_enemy), then use the generic "Assign Variable" block with a dot-notation Name field (new_enemy.image_blend) to reach that one specific instance. Corrected in Lesson 5's Handout and Key, and added as a named exception to Standards Rule 4 so a future contributor doesn't reintroduce the same mistake by over-applying "prefer the purpose-built block." Checked Lessons 3 and 13's similar-looking Set Instance Variable uses (targeting Self and Other, respectively, inside a Collision event) and confirmed both are legitimate - Self/Other are real, supported targets on that block; a Create-Instance-captured variable is the specific case it can't reach. No broader retrofit needed beyond Lesson 5 based on this check, but worth re-verifying if a similar spawn-and-tint pattern shows up in a not-yet-audited lesson.

Why 8 engines: The multi-engine structure (one shared weekly lesson, built out independently in Scratch, Godot, Unity, GameMaker, GDevelop, TIC-80, 001 Game Creator, and Unreal) is probably the book's core hook - it's an unusual structure for a game-programming book, most of which pick one engine. Worth an early chapter making the case for why explicitly: it demonstrates that the underlying concepts (loops, events, state, collision) are engine-agnostic, and lets a reader/student pick whichever engine matches how they think (text code, blocks, or pure visual wiring) without missing anything.

The Level 1/2/3 framework as the book's spine: MVP / Complete / Refined is used every single week for a small exercise, AND re-applied at the whole-semester level in Lesson 15 (MVP = Lesson 6, Complete = Lesson 11, Refined = Lesson 15). That fractal quality - the same

rubric working at two very different zoom levels - is a strong structural device for the book itself, not just the course.

The Structure goal as a running architecture throughline: Every lesson's Structure goal is doing the same underlying job (separation of concerns) but the book should resist presenting it as one repeated checkbox - it genuinely escalates week to week: co-locate-by-trigger, then object-owns-its-own-behavior, then spawner-vs-behavior, then unrelated-counters-stay-unrelated, then single-reusable-entry-point (DRY), then consolidate-a-system-built-piecemeal. Full breakdown already captured in Cross_Engine_Consistency_Reference.docx - cite it rather than re-deriving this in the manuscript.

Rule 5 origin story is good anecdote material: The Unreal Lesson 2/3/4/13 rebuild this session is a genuinely good illustrative story for a sidebar or pull-quote: Unreal's content had quietly drifted into flat-color Materials where every other engine used the real supplied sprite, close enough to work but not actually consistent - which is exactly what prompted Robert to articulate the Engine Deviation rule (Standards Rule 5) explicitly. Good real example of "a rule written after the mistake it prevents," which is honest and relatable for readers building their own multi-platform anything.

Engine-specific gotchas as troubleshooting sidebar content: Several hard-won, engine-specific bugs surfaced this project are strong candidates for "common pitfalls" sidebars rather than being filed away as pure production trivia: Unreal's Blank template auto-possessing a flying DefaultPawn and eating movement input; the "multiply a color parameter by white is a no-op" trap that bit Godot, Unity, GameMaker, AND Unreal independently for the Lesson 13 hit-flash; GameMaker/001GC's generic "destroyed" hook firing for both a hit AND an escape in Lesson 7, requiring an actual code restructure. These read well as "here's a mistake you'll probably also make, and why it happens" content.

Production philosophy worth explaining to readers, not just following internally: The Self-Containment rule (no lesson sends a student hunting for an asset mid-class) and the Purpose-Built-Blocks rule (use the engine's specific named block over a generic one, in visual scripting) both read as genuine craft opinions a book reader would want stated outright, not just inferred from the finished lessons. Consider a short "how these lessons were actually built" chapter or appendix drawing on Lesson_Production_Standards.docx.

Source documents - don't duplicate, cite: Lesson_Production_Standards.docx and Cross_Engine_Consistency_Reference.docx (both in Lessons/) are the primary source material for anything about course philosophy, cross-engine rationale, or production conventions. This notes document should stay focused on book-specific decisions, open questions, and raw commentary - not a third copy of the same rules.

Rule 6 retrofit scope: forward-only vs. retrofit: Standards Rule 6 (SuperStretch tie-in, a Refinement and Style pass, and visual-scripting node-reuse guidance) is a standing requirement now, but it wasn't in force when Lessons 1-15 were built, so none of the 15 completed Handouts/Keys currently carry it. Two options: (1) retrofit - go back through all 15 lessons x up to 8 engines and add Rule 6 content to the live, already-shipped Handouts/Keys now; (2) forward-only - apply Rule 6 starting with Lesson 16, and write the Rule 6 content for Lessons 1-15 fresh at book-writing time instead, since the book chapter is a separate deliverable from the live-class Handout and can carry more reflective, after-the-fact content anyway. Leaning recommendation: forward-only. The SuperStretch tie-in is cheap to backfill later - most of that content already exists in the standalone SuperStretch docs and just needs pulling into context. The Refinement-and-Style and node-reuse content is genuinely new writing,

at a scale (15 lessons x up to 8 engines) that would be a large side project for a live class with no real classroom benefit, since the students who used Lessons 1-15 have already moved past them. Better to write that material once, well, when the manuscript actually reaches those chapters, than twice - once retrofitted into a Handout already taught from, and again for the book. This is a recommendation, not a decision; Robert asked to scope this only and explicitly did not authorize starting the retrofit.

"Who's in charge" - a real Rule 6 case study, deliberately not retrofitted: Lesson 4's shipped Handout/Key (every engine) has one side of a collision - the Bullet in Unity/GameMaker/GDevelop/TIC-80, the Enemy in Godot - detect the hit and destroy BOTH objects. The cleaner alternative: Bullet and Enemy each detect their own hit independently and destroy only themselves, so neither object needs to reach into and manipulate the other. Robert built his own project this way and confirmed the decision explicitly: keep the simpler, common beginner pattern in the live Lesson 4 content on purpose (functional progress first, for a student still getting collision detection working for the first time), and save the "who's in charge" framing - already introduced verbally in class, not yet written down anywhere - for a Rule 6 Refinement and Style section or a dedicated Sidebar later. Good, concrete, real-classroom example of Rule 6 in action: recognizing a legitimate improvement mid-lesson and consciously choosing NOT to interrupt functional progress with it. A companion example from the same conversation: Unity's Level 3 Sound idea already avoids a sound getting cut off by Destroy() via AudioSource.PlayClipAtPoint() (a detached, self-cleaning audio source) rather than Godot's hide+await+queue_free() dance - Robert had used a plain AudioSource as a child of the object instead, which gets torn down along with its parent even though Destroy()/queue_free() are deferred to end-of-frame, not instant. Same root cause as the collision-detection case (an object's behavior depending on something outside itself), same fix (self-contained ownership) - a nice paired example for the same Sidebar or Refinement section.

Diagram pilot: Lesson 5 flowchart + GameMaker node wireframe: First use of hand-drawn branded diagrams outside Lesson 2's original set, prompted by Robert noting he keeps re-explaining the same logic verbally in class. Two new diagrams, built as branded SVG (matching the existing Brand_Assets/Diagrams navy/gold UML style) and rendered to PNG via cairosvg: diagram_spawn_loop.png, an engine-agnostic activity diagram of the counter/repeat-check/create/increment shape behind every engine's Level 1 solution this lesson, embedded into all 8 engines' Handout AND Teacher's Key (right before the Level 1 steps/code); and diagram_gml_visual_wireframe.png, a GameMaker-only mock-up of the actual GML Visual block layout (Instance - Create Instance's Target field feeding Variable - Assign Variable's dot-notation Name field), embedded into GameMaker's Handout Level 3 Visual-Track section and Key altNotes section only. Scoped this way per Robert's explicit answers: diagrams live in both the Handout and the Key (not just one); this is a pilot on Lesson 5 only, not yet a retrofit of Lessons 1-4/6-15 and not yet a standing Standards Rule; and node-wireframe diagrams are GameMaker-only for now, since it's the only engine with students actively switching between Script and Visual tracks - Unreal was deliberately excluded from wireframe scope since Robert's two Unreal students recently moved to lighter-weight engines. See the new ToDo item for the retrofit/standing-rule decision, which is intentionally not made yet.

Diagram pilot promoted to Standards Rule 9 - full Lesson 5 extension: Robert decided the pilot's outcome directly: diagrams become a standing rule (Standards Rule 9), not a Lesson 5-only experiment, and he expanded the scope in the same request rather than narrowing it. The Activity Diagram is now standard in every lesson's Handout, Key, AND Slide Deck (previously Handout+Key only). Engine node diagrams now cover all three dual-track engines - GameMaker, Unity, and Unreal - reversing the earlier GameMaker-only wireframe scoping once

Unreal's own student-enrollment status turned out not to matter for whether the diagram gets built. The Key always gets a node diagram (Level 1, plus a full combined-sequence diagram for Level 3 when the visual path is complex enough); the Handout always gets one for Level 1 specifically (added as an explicit follow-up note - previously Handout diagrams were novelty-gated only), plus an additional one when something genuinely unusual is introduced beyond Level 1 (the GameMaker back-door wireframe remains the model case for that half of the rule). Sidebars and Slides get diagrams opportunistically, when they'd clarify the text, not as a fixed requirement. Robert also flagged that a Slide Deck's diagram of a concept may deliberately not match an engine-specific document's diagram of the same idea, since engine-specific diagrams must authentically reflect that engine's actual visual paradigm (GameMaker's solid block-stack vs. Unity/Unreal's node-graph-with-pins) while Slides can simplify or generalize. Four new diagrams were built to extend Lesson 5 under the new rule: diagram_gml_visual_level1.png and diagram_unity_visual_scripting_level1.png (Level 1-only node diagrams for the Handouts), and diagram_unreal_blueprint_level1.png plus diagram_unreal_blueprint_full_sequence.png (Unreal's first node diagrams in this course, for both Handout and Key). All four wired into build_lesson05_handouts_keys_checklist.js's data-driven diagram hooks; the generic diagram_spawn_loop.png was added to the Slide Deck's Loops section; and diagram_unreal_blueprint_full_sequence.png plus the existing diagram_gml_visual_wireframe.png were added to the Lesson 5 Sidebars' "Script vs. Visual Coding" topic to reinforce its existing Unreal-vs-GameMaker instance-reference discussion. See Standards Rule 9 for the formal rule text and the ToDo items above for the still-open past/future retrofit work.

Correction: Apply To solves the GML Visual targeting problem after all: Robert found, through his own hands-on testing in GameMaker, a cleaner solution to the exact problem the earlier "GML Visual: Set Instance Variable can't target a dynamically-created instance" note (above) documented - and it changes that note's conclusion. The earlier note's finding that Set Instance Variable's Action Scope dropdown can't reach a variable-held instance reference was correct as far as it went, but its fix (a generic Assign Variable block with a typed dot-notation Name field, new_enemy.image_blend) turns out not to be necessary. The actual solution: drag an "Instance - Apply To" block right after Create Instance, and attach whatever needs the scoped target - an If/Else block, in this lesson's case - to the RIGHT of Apply To, not below it, which is what nests it inside Apply To's scope rather than just sequencing after it. Apply To's own Action Scope pop-up has an Expression field at the top, separate from its Self/Other/object list, and typing the same variable Create Instance's Target field captured (new_enemy) into that field re-points what "Self" means for every block nested inside. That means the actual purpose-built block - Set Instance Colour, not the more generic Set Instance Variable - works perfectly once nested inside Apply To's scope, with no generic block or typed workaround needed at all. Corrected in Lesson 5's Handout and Key, in Standards Rule 4 (the "confirmed exception" is now a resolved case), and in the Sidebars' Script vs. Visual Coding topic, which had built an entire GameMaker-vs-Unreal comparison around GML Visual allegedly having "no socket" for an instance reference - Apply To's Expression field is exactly that socket, just reached through a scope-setting block and a pop-up field rather than a draggable wire on the canvas. Both diagrams (wireframe and full sequence) were redrawn to match. Worth a general lesson for future audits: a purpose-built block's own targeting dropdown looking limited doesn't necessarily mean the block can't be used - check whether a scope-setting block like Apply To (or an engine's equivalent) changes what's available before concluding a generic/typed fallback is the only option.

Ternary operator pulled from Lesson 5's GameMaker code; save it for Semester 2: Lesson 5's GameMaker Code path originally set image_blend using a one-line ternary, (i % 2 == 0) ?

make_color_rgb(200, 25, 55) : make_color_rgb(230, 100, 130) - a pattern Robert genuinely likes and uses himself all the time, but judged a bit much to introduce to beginning students this early in the course. Rewritten as a plain if/else block instead, which is longer but reads unambiguously for a student who hasn't met conditional expressions as values yet. Robert's own suggestion: the Semester 2 capstone project - once students are comfortable with the fundamentals and are extending a more mature codebase - is a good, deliberate place to introduce the ternary operator formally, rather than leaving it to be picked up incidentally or never. Not yet scheduled to a specific Semester 2 lesson; see the paired ToDo item.

Demo-engine rotation drifted, then got re-planned as Standards Rule 10: Robert noticed Godot had been the live lecture-demo engine every single lesson from 3 through 14 (not just a few weeks running, as he'd first suspected) and asked for a full per-lesson list, which surfaced the drift. A scan back through the project turned up real evidence that a rotation had genuinely been pre-planned, not just remembered inaccurately: the Syllabus, Engine & Platform Guide, and Engine Showcase docs all promise students an even rotation across engines/styles, and the 30-Lesson Master Schedule spreadsheet has an actual Featured Engine + Coding Style column for all 30 lessons modeling that balance. The promise was simply never enforced as a Standards Rule, so nothing stopped a lesson's "continue last week's demo project" framing from perpetuating whatever engine got picked once. Since Lessons 3-15's actual topics have diverged from the master schedule's original per-week order, the old schedule couldn't just be restored - a new rotation had to be designed against the lessons as actually built. Robert then narrowed the pool to only the engines/styles with active students (Scratch, GameMaker script, GameMaker visual, Unity C#, Godot), deferring full 8-engine/3-style balance to a ToDo item (above) rather than designing for engines nobody's using yet. Finalized rotation for Lessons 5-14, each choice appearing exactly twice, no back-to-back repeats, preserving a "same engine, two modes" GameMaker callback pairing (mirroring the original schedule's design intent): L5 GameMaker Visual (+ brief script contrast, ties to the Script-vs-Visual Sidebar), L6 Scratch, L7 Godot (unchanged), L8 Unity C#, L9 GameMaker script (callback to L5), L10 Godot (unchanged), L11 Unity C#, L12 GameMaker Visual (callback to L5), L13 Scratch, L14 GameMaker script (callback to L9). L15 stays demo-free (workshop/showcase day) per the original design. Every affected lesson's Plan (Featured Engine field), Teacher's Script (demo cues and talking points), and, for Lesson 5 only, Slide Deck speaker notes were rewritten to be technically accurate to their newly assigned engine - not just relabeled.

Enemy-vs-enemy self-collision: a real bug, found engine by engine, fixed quietly rather than taught yet: Robert hit a genuine runtime bug testing Lesson 5 Godot himself: no enemies appeared on screen or in the Remote Scene tree during play. Diagnosis (his own testing plus a review of the shared Hitbox/area_entered collision architecture from Lessons 3-4) found the real cause: Godot's Hitbox area_entered handler has no type-check on what triggered it, and at the original 40px-apart / 60px-spacing layout with 128x128px sprites, adjacent Enemies' hitboxes overlapped at spawn and mutually destroyed each other before the first frame was ever seen - confirmed directly by Robert reducing the loop count and watching enemies survive, then hearing the destroy sound fire once the count went back up. His chosen fix, deliberately the narrow one: widen spacing only (Vector2(150 + i * 200, 50)), explicitly declining the deeper architectural fix (a real Collision Layer/Mask setup, or a type-check in the handler) for now. Robert then reported Scratch has the identical problem - overlapping clones causing double-hits rather than full destruction - which he'd already independently fixed himself (enemy_i renamed to enemy_number, spacing formula changed to enemy_number * 100 - 300) before this was ever raised as a course-wide question. That prompted a full audit of the remaining six engines for the same generic-trigger-fires-on-any-overlap pattern: GameMaker is architecturally immune (its Collision Event is object-pair-specific, not generic); Unity (OnTriggerEnter2D +

CompareTag), Unreal (Cast To BP_Bullet), GDevelop (named-pair collision condition), and TIC-80 (hand-coded rectangle-overlap math) are all safe by design for the same reason. Only 001 Game Creator was found vulnerable (its "Collided with actor" trigger is generic, with no actor-type check) - fixed the same way as Godot and Scratch, by strengthening the placement instructions in Lessons 5 and 14 ("at least a full enemy-width of empty gap") rather than restructuring the trigger logic. Robert was explicit about scope throughout: fix the practical problem silently, everywhere it exists, right now - but do not yet teach collision-type filtering (Bullet vs. Player vs. Enemy vs. Enemy) as a formal topic. A review of Lessons 6-15's actual content confirmed there was no orphaned teaching moment being skipped by this choice: true Player-vs-Enemy touch collision never appears anywhere in Semester 1 (only Bullet-vs-Enemy from Lesson 4, and an escape-based life-loss mechanic in Lesson 8), and Lesson 15's own teacher script already frames collision-type filtering as Semester 2 territory. Good candidate for a Semester 2 sidebar or lesson-opener later: Robert's own framing for it, already used verbally in class per his message, was "Now our enemies are moving around, they can collide with each other AND the player. How do we fix this? What even is the right action we want for each type of collision?"

Cross-engine consistency promoted from habit to Standards Rule 12: While retrofitting Unity's Lesson 5 code to Rule 11 (meaningful variable names), a real naming conflict surfaced: Unity's EnemySpawner already has a public enemyCount field (the total number of enemies to spawn), so reusing that same name for the loop counter - the literal move Rule 11 alone would suggest, matching GameMaker and Godot's enemy_count - would shadow it, technically legal in C# but genuinely confusing. The loop counter was named enemyIndex instead, a distinct and still meaningful name, and the mismatch was flagged to Robert rather than resolved unilaterally. Robert's response went beyond just answering that one question: leave the enemyCount/enemyIndex mismatch alone for now, but treat the underlying question - keeping variable names AND code structure consistent across all 8 engines - as a fundamental precept of the course, not a nice-to-have. His reasoning, stated directly: he has personally observed students helping each other across engine and coding-style lines - a student on one engine comparing notes with a student on another, working through the same week's exercise - and the more common ground the actual names and structure share, the easier that peer help is. A student has to mentally translate "my enemy_count" to "their enemy_number" before they can even start comparing approaches; consistent naming and structure removes that translation tax entirely. This was true from the start of the multi-engine design but wasn't enforced strictly early on (Lessons 1-4 predate any naming rule at all, and Lesson 5's own Rule 11 fixes were applied engine-by-engine as each was caught rather than as one coordinated pass) - Robert is now retroactively naming it as something the course has valued implicitly all along, now that enough real classroom evidence exists to state it as a rule outright. Formalized as Standards Rule 12; a dedicated future cross-engine reconciliation pass (covering both naming and structure, across all lessons and engines) is tracked as a ToDo item above, deliberately scheduled later rather than forcing an immediate resolution of the Unity case that surfaced it.

Data Dictionary project launched to drive the Rule 12 consistency goal: Immediately after Rule 12 was formalized, Robert asked for a central, formal Data Dictionary - not just a ToDo describing the consistency goal, but an actual working reference naming every variable, function, and file the course creates: one common cross-engine concept name (e.g. "per-frame update"), paired with each engine's own actual name for that same concept where engines can't literally share one identifier. Undecided yet whether this will ever be shown to students, or stays a production-only tool - built either way, since it's useful internally regardless. Robert's first-choice format was a genuine Microsoft Access database with a data-entry Form and a Report, explicitly because he dislikes working in spreadsheets. Checked before building anything:

Access's .accdb format (2007+) has no Linux-native authoring path at all - mdbtools, the standard Linux toolkit for Access files, doesn't even read/write the modern .accdb format (it's limited to the legacy Jet 3/4 .mdb format from Access 97/2000/2002), and more fundamentally, Access Forms and Reports are proprietary, VBA-hosted application objects with no file-level equivalent outside Access itself - no export format, no library, nothing short of the real Access application (Windows-only, not available in this sandboxed environment) can author a working Form or Report object, regardless of which .mdb/.accdb table format is targeted. Confirmed via both a direct tool check and a web search before ruling it out, rather than assuming. Robert had already pre-approved the fallback ("I'll live with an Excel spreadsheet but I'll pout about it"), so the dictionary is being built in Excel instead, with an explicit effort to make it feel less like a bare spreadsheet than the default - data-validation dropdowns, a proper per-engine reference layout, and the closest practical approximation of the Form/Report feel Excel supports. Same conversation also raised the open case/style question (PascalCase vs. camelCase vs. snake_case) as a Rule 12 sub-decision, with Robert's stated leaning toward prioritizing cross-engine consistency in the chosen case style over each engine's own local idiom, explained via Sidebar wherever the two disagree - see the paired ToDo entry, not yet a final decision.

GameManager: a dedicated shared-state object, promoted from a Unity-only accident to Standards Rule 13: Reviewing Lesson 7 surfaced an asymmetry that had been sitting unnoticed since Lesson 6: Unity's Handout/Key already routed the win-counter (enemies_remaining) and win-flag (has_won) through a small dedicated GameManager object, while every other engine had that same shared state living wherever it happened to land - the spawner in some engines, a collision branch in others. This wasn't a deliberate Unity design choice being withheld from the rest of the course; it traced back to how Unity's Lesson 6 content happened to get written, and nobody had asked "why does only one engine do this" until Robert did. Once asked, the answer was obvious: a dedicated object whose only job is holding game-wide information is the better habit, full stop - Unity had simply stumbled into the right pattern first. Formalized as Standards Rule 13 (a dedicated manager object for any state more than one other object needs to read or change), and retrofitted into Lesson 6 across Scratch (a hidden GameManager sprite driven by broadcast/receive), Godot (a child GameManager Node reached via signal connection), GameMaker (obj_gamemanager, a placed instance), and Unreal (BP_GameManager, a placed Actor with a dispatcher binding) - Unity needed no change, having already modeled the target pattern. Good real-course illustration for the book of a rule discovered by noticing an inconsistency rather than designed up front - the opposite direction from Rule 5's origin story (a rule written after catching a mistake), this one was a rule written after catching an accidental correct answer nobody had generalized yet.

Lesson 7 compliance pass: GameManager routing, naming drift, and the full diagram/prose standard: With Rule 13 in place, Lesson 7 itself needed the same GameManager routing applied to its own win-tracking logic (all engines), plus a Teacher's Script callout telling students to go retrofit their own Lesson 6 project before starting Lesson 7's lab work, so the fix lands in every student's actual codebase rather than staying a documentation-only change. Auditing Lesson 7 against the full current Standards Rule set while making that change turned up two smaller drifts worth recording: Rule 11 (meaningful names) had regressed in a couple of spots - Scratch's enemies_left/enemy_i and Godot/GameMaker/Unity's bare speed field all got renamed to match the enemies_remaining/enemy_number/enemy_speed conventions established elsewhere - and Rule 9 (diagrams) hadn't been applied to Lesson 7 at all yet. Five new diagrams were built and wired into every Handout, Key, and the Slide Deck: a generic Activity Diagram contrasting the two ways an enemy can disappear (escape vs. hit, only one of which should decrement the counter - the exact bug this lesson's exercise centers on), a State Diagram for the enemy's two possible endings, and Level-1 node diagrams for the three

dual-track engines (GameMaker GML Visual, Unity Visual Scripting, Unreal Blueprint). Unity's Visual Scripting altPath prose was also tightened afterward to name each node exactly as the diagram draws it (Update -> Translate -> Branch -> Destroy) rather than compressing the graph into a single dense sentence, matching the per-node granularity Lesson 5 set as the specificity bar for alt-path writing. Unreal's Blueprint path needed no equivalent prose tightening - it was already written at full specificity as the lesson's PRIMARY path (not an altPath), and the audit's original complaint about it (no diagram backing the prose) was resolved by the new diagram set.

Lesson 8 compliance pass: GameManager retrofit, naming fixes, and the full diagram/notes standard: Lesson 8 (the lives/lose-condition lesson - the mirror image of Lesson 6's win condition) got the same treatment as Lesson 7, this time against a slightly different set of gaps. Rule 13: Lesson 8's lives counter and lose-flag (has_lost) were retrofitted onto each engine's GameManager object (mirroring exactly how enemies_remaining/has_won already route through it since the Lesson 6/7 retrofit), rather than living on the player or a collision handler directly - the same routing pattern, applied to the opposite condition. Rule 11/12: a genuine TIC-80 naming regression was caught and fixed (the bare j/e loop variables flagged as a known gap in the standing ToDo list finally got their Lesson 8 instance renamed to enemy_index/enemy, matching the Lesson 7 precedent), alongside a small Scratch naming drift back toward an earlier inconsistent variable name. Rule 7: GameMaker's Draw GUI altPath and Unity's Visual Scripting altPath both had field-value and node-name gaps tightened to full explicitness (exact Draw Text field values spelled out; Unity node names matched one-for-one to what the new diagram draws), closing the same category of gap Lesson 7's Rule 7 audit had already fixed elsewhere. Rule 9: a full diagram set was built and wired in - one generic Activity Diagram (the lose-condition logic, shared across every engine's Handout/Key/Slide Deck) plus three engine-specific Level 1 node diagrams (GameMaker GML Visual, Unity Visual Scripting, Unreal Blueprint), wired into every Handout and Key. This lesson surfaced a placement nuance not previously written down: in the Handout, the node diagram nests under the altPath for engines whose primary documented path is code (Unity, GameMaker), but sits as a top-level diagram for Unreal, whose primary path is Blueprint itself, not an alt; in the Key, the node diagram is always top-level for all three dual-track engines, since the Key presents the full combined solution rather than a split main/alt track. Rule 10/8: the Slide Deck had drifted to reference Godot in its live-demo materials list and Slide 9's content box, left over from before the Lesson 5-14 rotation was finalized (Lesson 8's assigned engine is Unity C#) - corrected to Unity/GameManager.RegisterEscape() throughout - and had zero addNotes() calls despite Robert's classroom now running Presenter View, so six speaker-notes entries were added (slides 5, 7, 9, 11, 13, 14 - matching the Lesson 6 reference pattern of skipping title/overview/divider slides), each carrying that slide's matching Teacher's Script section verbatim, including bracketed stage directions and timing checkpoints.

Lesson 9 compliance pass: score joins the GameManager, a Unity Rule 7 gap, and the full Rule 9 diagram/Rule 8 notes treatment: Lesson 9 ("On the Scoreboard" - score tracking plus a persistent on-screen HUD, introducing the Push vs. Poll display-update concept) got the same full audit-then-fix treatment as Lessons 7 and 8, requested proactively by Robert while he personally reviewed Lesson 8's materials rather than in response to a specific bug report. Rule 13 was the single biggest finding: Lesson 9 had fully regressed on the GameManager pattern across four engines, each in a different way - GameMaker was routing score through obj_spawner instead of obj_gamemanager; Scratch had the Enemy sprite's own script changing score directly instead of staying confined to GameManager's existing "when I receive [enemy hit]" broadcast handler; Godot had score and a new update_hud() function living on "the main scene's script" instead of GameManager's script, breaking the established get_node("../ScoreText")-from-GameManager pattern; and Unreal had Score and the HUD

widget setup living on the Level Blueprint instead of BP_GameManager, the same Actor that already owns EnemiesRemaining and Lives. All four were corrected to route score (and, for Godot/Unreal, the new push-based update function) through the established manager object, cross-checked line-by-line against Lesson 8's already-compliant content as the reference baseline for each engine's correct object/function names rather than improvising a new pattern. Two smaller drifts were caught and fixed opportunistically along the way: a Rule 11/12 Scratch naming regression (enemies_left had crept back in, corrected to enemies_remaining), and a Rule 7 gap in Unity's Visual Scripting altPath (vague "a node" references tightened to exact node names and categories - Format String/String category, Add/Math-Scalar category - matching the specificity bar Lessons 5 and 8 established). Rule 9: a full diagram set was built - one generic Activity Diagram ("Score Joins the Hit Path - Then Push or Poll Takes Over," covering both the GameManager routing and the Push-vs-Poll architectural choice in one picture) plus three engine-specific Level 1 node diagrams (GameMaker GML Visual, Unity Visual Scripting, Unreal Blueprint) - and wired into every Handout and Key following the Lesson 8 precedent exactly: nested under the altPath in the Handout for GameMaker and Unity (whose primary path is code), top-level in the Handout for Unreal (whose primary path is Blueprint), and always top-level in the Key regardless of Handout placement, since the Key shows the combined solution. Rule 10/8: the Slide Deck's materials list and Slide 9 content box had drifted to reference Godot, left over from before the Lesson 5-14 rotation was finalized (Lesson 9's assigned engine is GameMaker script, a deliberate callback to Lesson 5's GameMaker Visual demo) - corrected to GameMaker/obj_gamemanager.score throughout - and had zero addNotes() calls, so six speaker-notes entries were added (slides 5, 7, 9, 11, 13, 14, the same pattern Lessons 6 and 8 established), each carrying that slide's matching Teacher's Script section verbatim.

GameMaker drag-and-drop uses instance variables (__dnd_lives, __dnd_score) - a correction found by testing: While reviewing Lesson 8 for GameMaker, Robert noticed that the Visual Scripting content relied on deprecated built-ins: the old global lives/score/health have been replaced by drag-and-drop's own instance-level variables, __dnd_lives and __dnd_score (dnd = drag and drop), and purpose-built blocks exist for them (Set Lives, If Lives, Draw Instance Lives, Set Score, Draw Instance Score), each with a Target drop-down that must point at the GameManager object. The earlier content had wrongly used generic Set Instance Variable blocks with Lives/Score as a supposed recognized built-in. The first proposed fix (keep a custom variable in code and leave the code and block paths separate) was rejected in favour of making the code path use the very same __dnd_ variables, so a student can mix typed code and blocks in one project. Lessons 8 and 9 were corrected on that basis; Lesson 8 also gained the missing sound assets (sfx_lifelost, sfx_gameover, sfx_start) so students never have to find their own. Good book anecdote: the author's own hands-on testing found what documentation-driven drafting missed, and the same session surfaced follow-up questions (dotted obj.variable vs. Target drop-down, Prefabs, asset-name prefixes) now tracked in the To-Do list.

GML Visual has no Else If block - a genuine engine-specific gotcha, not yet retrofitted: Robert's observation: GameMaker's GML Visual (drag-and-drop) toolbox appears to have no "Else If" node/block at all, unlike the code path's natural else if chain. The only way to express a third (or later) branch of a multi-way decision in Visual is to nest another If-style block one level deeper, inside the previous block's Else slot - so a three-way decision isn't three same-level blocks in a row, it's an If, whose Else contains another If, whose Else contains the final branch, and so on for each additional case. This is exactly the kind of engine-specific gotcha the project has already flagged as good troubleshooting-sidebar material elsewhere (see the earlier note on engine-specific gotchas as a candidate for a "common pitfalls" chapter): a student who's only ever written else if in text code will very reasonably go looking for an Else If block in

GameMaker Visual, not find one, and assume the tool can't do what they want rather than realizing the shape is just different. Not yet checked against the shipped course content - see the matching To-Do item for which lessons (Lessons 8, 9, and 11 are the likely candidates, anywhere a lesson's logic needs to tell apart three or more outcomes - lives hitting zero vs. enemies hitting zero vs. neither, or a wave clearing vs. the whole round being won) actually have a three-or-more-way branch in their GML Visual content, and whether that content already nests correctly or was written as if a flat Else If existed.

Debugging 101 / Testing 101 / Testing-Debugging 201 - a companion course family, spitballed from this project's own bug history: Robert's idea, raised as a sideways thought rather than a scoped proposal: a new book/class, "Debugging 101," built around a code repository pre-loaded with a variety of deliberately planted bugs for students to find and fix. The named bug categories he gave map with unusual precision onto bugs this very project actually produced and recorded: flat-out broken code that doesn't compile or run; a failed edge case (the Stalemate is exactly this - kills and escapes are each handled correctly in isolation, but the combination where some of each happen in one round was never considered); a new feature breaking an existing one (the dropped Level 3 sounds - a Rule 14 "complete method" regeneration for a later lesson silently deleted working Level 3 code from an earlier one); a mismatch between a design doc and the actual code (the WaveAnnouncementText/WaveText naming mismatch between a Handout's prose and its Teacher's Key); a broken refactoring (the GameMaker Visual targeting saga - a generic block was swapped for another generic block, then finally a purpose-built one, with two of the three attempts subtly wrong in a way only hands-on testing surfaced); and a misinterpreted feature (built to a literal reading of a spec, but not to what was actually intended - the enemy-vs-enemy self-collision bug is a plausible fit here too, where a generic "anything touched this" trigger was technically working as coded but not as meant). The parallel idea, "Testing 101": the inverse situation - an existing codebase plus a design doc, but no test suite at all, where the student's job is to write the tests. A possible synthesis, "Testing/Debugging 201": combine both so that writing a test case IS the mechanism that proves a fix actually worked, not just a separate skill taught alongside debugging - red test before the fix, green test after. None of this is scoped: no decision yet on audience (same Middle/High School homeschool audience as Video Game Creation, or a different one entirely), language/engine (the bug-category list reads as language-agnostic, but the actual repository would need a concrete stack), length (a standalone book, a short course, a unit folded into an existing one), or relationship to the main course (standalone companion vs. a Semester 2/Year 2 extension of Video Game Creation itself, given how directly its source material comes from this project). Recorded here purely so the idea isn't lost before it's had time to develop; revisit once Semester 1 of Video Game Creation is further along or complete.

Revision Note

First entry, written the same week as the Unreal Lesson 2/3/4/13 rebuild and the first pass of Cross_Engine_Consistency_Reference.docx. Updated the same week to add Standards Rule 6 (SuperStretch/Refinement and Style/visual-scripting reuse) and scope, but not begin, the question of retrofitting it into Lessons 1-15. Updated again to add Standards Rule 13 (dedicated manager object for shared state), the Lesson 6 GameManager retrofit, and Lesson 7's full compliance pass (GameManager routing, Rule 11 naming-drift fixes, the new Rule 9 diagram set, and Unity Visual Scripting prose tightening). Updated again to add a GameManager mini-lecture to Lesson 7's live slide deck (placed at the start of the lab section, separate from the main lecture) plus the paired ToDo to relocate that discussion to Lecture 06 once the book manuscript reaches it. Updated again to add Lesson 8's own full compliance pass (the

lives/has_lost GameManager retrofit, Rule 11/12 TIC-80 and Scratch naming fixes, Rule 7 visual-path explicitness fixes, a new Rule 9 diagram set with a newly-documented Handout-vs-Key placement convention for the three dual-track engines, and the Rule 10/Rule 8 slide deck fixes correcting a Godot-leftover demo reference and adding its first six speaker-notes entries), and to narrow the Rule 7/8/9/13 retrofit ToDo items' remaining scope accordingly. Updated again to add Lesson 9's own full compliance pass (a four-engine Rule 13 GameManager retrofit for score - GameMaker, Scratch, Godot, and Unreal - plus a Rule 11/12 Scratch naming fix, a Rule 7 Unity Visual Scripting explicitness fix, a new Rule 9 diagram set reusing the established Handout-vs-Key placement convention, and the Rule 10/Rule 8 slide deck fixes correcting a Godot-leftover demo reference and adding its six speaker-notes entries), and to narrow the Rule 7/8/9/13 retrofit ToDo items' remaining scope to Lessons 10-15 accordingly. Updated again to record the GameMaker __dnd_lives/__dnd_score correction (Lessons 8-9) and five follow-up To-Do items (Lessons 10-15 review, obj.variable vs. Target drop-down, an If Lives comparison sidebar, GameMaker Prefabs, and dropping asset-name prefixes). Updated again to record the Unity static-members items, the cross-engine name normalization To-Do and the canonical bug-hunt decision, plus the Lesson 10 compliance pass and the cross-engine design-drift To-Do (HUD object, prefabs). Updated again at the Lesson 11 review to point to the new Reflex Habits design-notes document and to record the name "the Stalemate" for the win/lose gap bug and the Lesson 11 compliance pass (active engines only). Updated again to record a spitballed new companion course family - Debugging 101, Testing 101, and a combined Testing/Debugging 201 - raised by Robert as a sideways idea, not yet scoped. Updated again to record the GML Visual "no Else If block" gotcha (not yet audited against shipped content) and to extend Standards Rule 12's scope to cross-engine pop-up/HUD text presentation (centering, placement, size), with reasonable shared defaults proposed but not yet decided or audited. Add to this document as the course continues; do a dedicated consolidation pass every so often rather than continuously.